Arithmetic Expression Evaluator in C++

History Module Test Cases

Version 1.0

Revision History

Date	Version	Description	Author
02/May/2026	1.1	First addition of test cases	Courtney
03/May/2026	1.2	Edited test cases, fixed formatting/document presentation	Courtney

Table of Contents

Test Cases

1. Purpose

This Test Case Specification document for the Calculatorz's History Manager module defines test cases used to verify the module correctly stores, retrieves, records, and clears calculation histories in the calculator application.

2. Successful Save of Calculations

2.1 Test case identifier

TC-HM-001

2.2 Test item

Tests the saveCalculation() function to ensure a calculation is correctly appended to the history file.

2.3 Input specifications

The filename used is "history.txt", the expression used is "2+2", and the result is "4".

2.4 Output specifications

A new line is added to the file in the following format: "YYYY-MM-DD HH:MM:SS | 2 + 2 = 4".

2.5 Environmental needs

This module will only be successful with a writable file system and a existing and creatable history file.

2.6 Special procedural requirements

Ensure the file is writable before running the test case.

2.7 Intercase dependencies

None.

3. Append Multiple Calculations

3.1 Test case identifier

TC-HM-002

3.2 Test item

Verifies that multiple calls to saveCalculation() appends new entries without overwriting existing data saved in the module.

3.3 Input specifications

The sequence of inputs for this test case will begin with “5*5” with a result of “25” and will be followed by a second input of “10/2” with a result of “5”.

3.4 Output specifications

The output should have two lines in the history file as follows:
“[timestamp] | 5 * 5 = 25
[timestamp | 10 / 2 =5”.

3.5 Environmental needs

An existing writable file.

3.6 Special procedural requirements

A special procedural requirement includes beginning with an empty history file.

3.7 Intercase dependencies

This test case depends on the success of test case TC-HM-001 and its ability to correctly save calculations.

4. Retrieve Last N Calculations

4.1 Test case identifier

TC-HM-003

4.2 Test item

Tests the ability of getLastCalculations() to ensure only the requested number of recent entries is returned to the user.

4.3 Input specifications

For example, if the history file contains 5 previous entries, call “getLastCalculations(3)”.

4.4 Output specifications

The output in this example should print a vector containing the most recent three calculations entered in the correct order (newest to oldest).

4.5 Environmental needs

A prepopulated history file with 5 previous entries.

4.6 Special procedural requirements

Ensure the history file contains at least 5 previous entries.

4.7 Intercase dependencies

This test case depends on the ability of TC-HM-002 to save multiple calculation entries in the correct order and without writing over one another.

5. Retrieve All Calculations When Count Exceeds File Size

5.1 Test case identifier

TC-HM-004

5.2 Test item

Verifies behavior when requested count exceeds number of stored entries.

5.3 Input specifications

A history file containing 2 entries and call “getLastCalculations(10)”.

5.4 Output specifications

Function returns both previous entries without error.

5.5 Environmental needs

A prepopulated history file is required for this test case.

5.6 Special procedural requirements

Ensure that only 2 previous entries exist within the history file before beginning this test.

5.7 Intercase dependencies

This test case depends on the success on TC-HM001’s ability to successfully save calculations

6. Retrieve History from Missing File

6.1 Test case identifier

TC-HM-005

6.2 Test item

Verifies that the getLastCalculations() function safely handles a missing history file without crashing the program.

6.3 Input specifications

Using a nonexistent filename, call “getLastCalculations(5)”.

6.4 Output specifications

Returns an empty vector: “{}”.

6.5 Environmental needs

The file must not exist before beginning testing.

6.6 Special procedural requirements

Delete or rename history file before execution.

6.7 Intercase dependencies

None.

7. Clear History Successfully

7.1 Test case identifier

TC-HM-006

7.2 Test item

Tests the clearHistory() function o ensure all previous history entries are removed.

7.3 Input specifications

History file that contains several calculation entries.

7.4 Output specifications

The history file will become empty but still exists after calling the function.

7.5 Environmental needs

A writable file system.

7.6 Special procedural requirements

Verify the file has previous calculation entries before completing the test.

7.7 Intercase dependencies

This test case depends on TC-HM-002 and its ability to properly save multiple entries to the history file.

8. Save Calculation File Open Failure

8.1 Test case identifier

TC-HM-007

8.2 Test item

Tests the exception handling when the history file cannot be opened for writing.

8.3 Input specifications

The input would include an invalid filename location and the expression “3+3” with the result of “6”.

8.4 Output specifications

The program should return “runtime_error(“Failed to open history file for writing)”.

8.5 Environmental needs

Requires a read-only directory or an invalid path.

8.6 Special procedural requirements

Restrict write permissions before testing.

8.7 Intercase dependencies

None.

9. Clear History File Open Failure

9.1 Test case identifier

TC-HM-008

9.2 Test item

Verifies exception handling when clearHistory() cannot access the given file.

9.3 Input specifications

An invalid or unprotected filename.

9.4 Output specifications

The program returns “runtime_error(“Failed to clear history file)”.

9.5 Environmental needs

A read-only directory or an invalid path is required.

9.6 Special procedural requirements

Restrict file permission before executing the test case.

9.7 Intercase dependencies

None

10. Ignore Empty Lines in History File

10.1 Test case identifier

TC-HM-009

10.2 Test item

Verifies that empty lines in the history file are ignored while being read.

10.3 Input specifications

A history file that contains at least one empty line like as follows:
“[timestamp] | 1 + 1 =2

[timestamp] | 2 + 2 = 4”.
This file goes along with the following call: “getLastCalculations(5)”.

10.4 Output specifications

Returned vector only contains the two valid entries from the history file.

10.5 Environmental needs

An editable history file is required to add empty lines.

10.6 Special procedural requirements

Manually inserting blank lines into the editable history file.

10.7 Intercase dependencies

This test case relies on TC-HM-002 for storing multiple calculation entries to history as well as TC-HM-004 and its ability to return all of the valid entries when the count exceeds the number contained in the history file.

11. Verify Timestamp Format

11.1 Test case identifier

TC-HM-010

11.2 Test item

Tests the formatting of the getCurrentTimestamp() function.

11.3 Input specifications

Call the “getCurrentTimestamp()” function.

11.4 Output specifications

Returns a string in the following format: “YYYY-MM-DD HH:MM:SS”. For example, “2026-05-02 10:52:36”.

11.5 Environmental needs

A valid system clock is required to track the current date and time for the timestamp.

11.6 Special procedural requirements

Compare the function output with the true date and time of the execution and with the desired format.

11.7 Intercase dependencies

None.